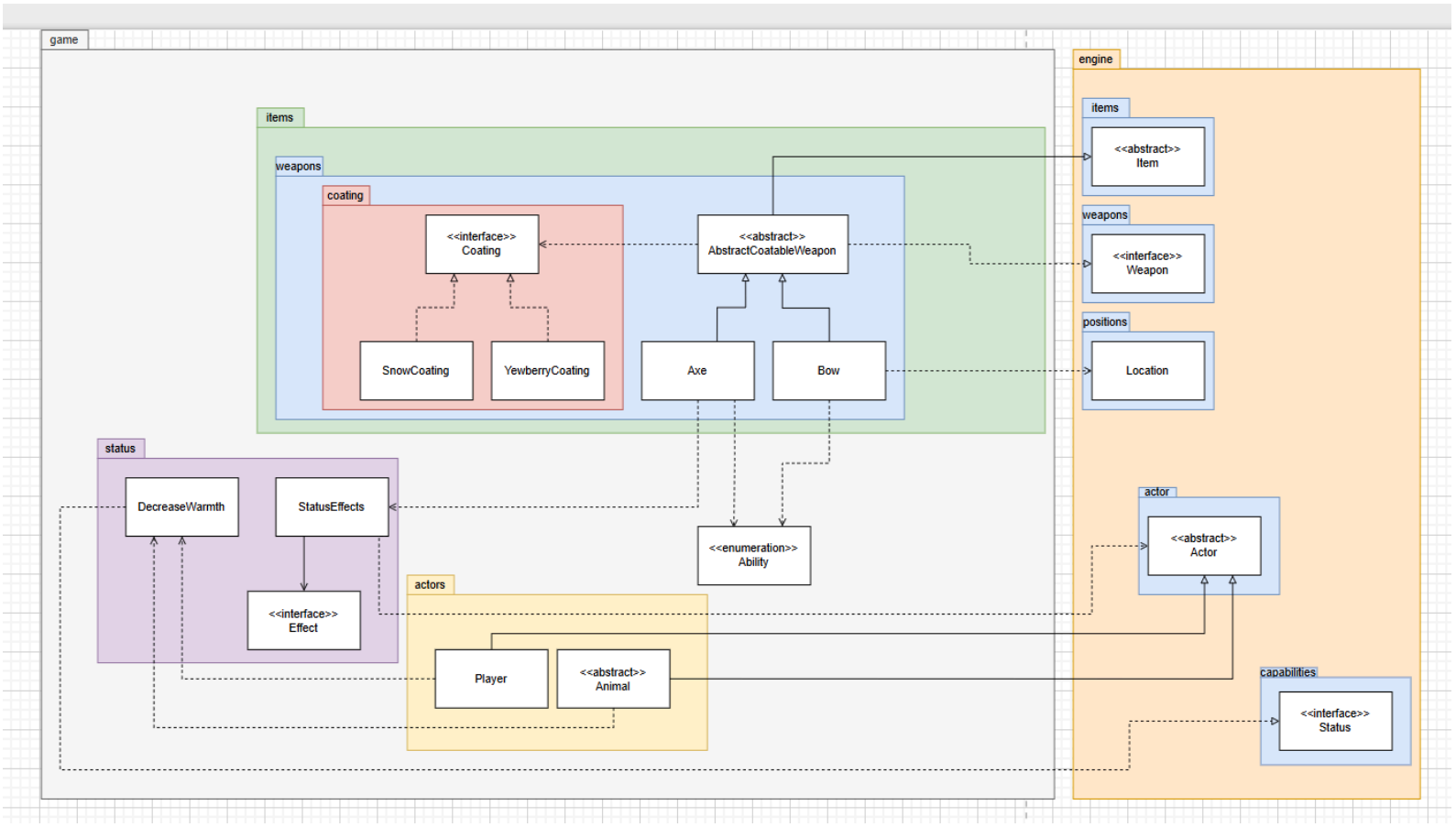
Design 4 UML diagram



Design Rationale

A. Implementing Coatable Weapons

Design 1:
Each weapon class (e.g., Bow, Axe) individually implements its own coating logic.

Pros	Cons
Straightforward since coating behavior is implemented directly inside each weapon class.	Code duplication as coating-related logic (e.g., apply, remove, replace coating) must be reimplemented in every weapon class.
Easy to modify coating behavior for a single weapon without affecting others.	Difficult to maintain and extend because adding new weapons requires rewriting the same coating logic.
No need for extra abstraction layers.	Violates DRY (Don't Repeat Yourself) principle and reduces scalability if more coatable weapons are introduced in the future.

Design 2:
An abstract class AbstractCoatableWeapon extends Weapon and implements common coating-related logic, while concrete classes like Bow and Axe extend it.

Pros	Cons
Improves reusability since all coatable weapons inherit common behavior from one place.	Slightly increases complexity due to the introduction of an abstract superclass.
Enhances scalability because future coatable weapons can be easily added by extending the abstract class	Non-coatable weapons must avoid extending this class, requiring careful inheritance planning.
Encourages clean architecture by separating general weapon logic from coating-specific logic	May require refactoring of existing weapon hierarchy if additional inheritance constraints exist.
Reduces code duplication as shared coating functionality (e.g., apply/remove coating) is centralized.	

B. Implementing Coating Behavior

Design 1:
Coating logic is handled directly inside the weapon or actor classes using conditional statements (e.g., if coating == "snow" …).

Pros	Cons
Simple and direct — easy to understand for small-scale implementation.	Poor scalability as adding new coatings requires modifying multiple classes.
No need for additional interfaces or classes.	Difficult to manage stacking and effect durations as logic is tightly coupled to weapon/actor classes.
Works fine when there are only one or two coating types.	Violates Open-Closed Principle because each new coating requires modifying existing code.

Design 2:
Define a Coating interface with methods such as onHit() and name(). Classes like SnowCoating and YewberryCoating implement this interface to provide specific effects.

Pros	Cons
Promotes extensibility — new coatings can be added easily without modifying existing code.	Slightly higher setup complexity due to multiple classes/interfaces.
Improves flexibility since each coating type encapsulates its own unique effect logic.	May require coordination between weapon and actor classes to integrate coating behavior correctly.
Encourages modular design, following the Open-Closed and Single Responsibility Principles.	
Cleaner and more maintainable structure as coating behaviors are decoupled from weapon logic.	

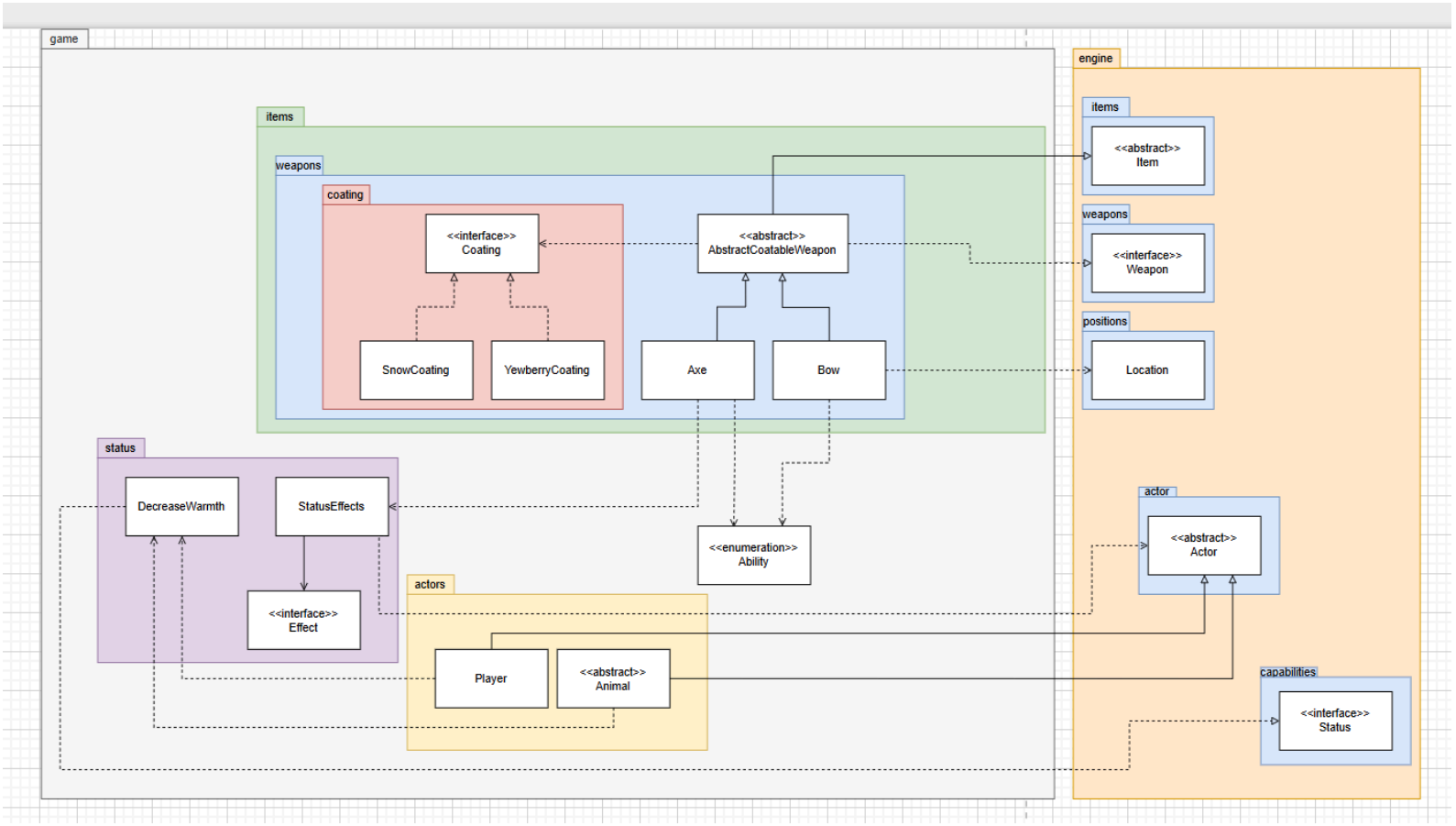
C. Managing Coating Effects on Actors

Design 1:
Handle status effects (poison, frostbite) directly in the Actor class.

Pros	Cons
Simple and intuitive to implement since all actor effects are managed in one place.	Makes the Actor class large and harder to maintain as more effects are added.
Quick access to actor stats and health during effect application.	Breaks the Single Responsibility Principle because the Actor now handles both gameplay behavior and effect management.
No additional classes needed.	Difficult to extend if new status effects or complex stacking rules are added.

Design 2:
Introduce a dedicated StatusEffects class to manage and apply effects such as poison and frostbite.

Pros	Cons
Improves modularity — all effect logic is isolated from the Actor class.	Slightly increases complexity as it introduces another class and requires coordination with the Actor.
Enhances scalability since new effects can be added without modifying the actor logic.	Requires clear communication (method calls or references) between Actor and StatusEffects.
Promotes cleaner, more maintainable code through separation of concerns.	
Facilitates effect stacking and duration tracking through centralized management.	



The diagram above illustrates the final design for Requirement 4, which adopts Design 2 from all three parts (A, B, and C). This design introduces the AbstractCoatableWeapon abstract class, the Coating interface (implemented by specific coating types such as SnowCoating and YewberryCoating), and a dedicated StatusEffects class to manage ongoing effects like poison and frostbite.

By introducing AbstractCoatableWeapon, coating-related logic is centralized and reused across all weapon types such as Bow and Axe, effectively achieving the DRY (Don't Repeat Yourself) principle. This abstraction reduces code duplication, simplifies maintenance, and allows for consistent coating behavior across multiple weapons. The design also adheres to the Open-Closed Principle from SOLID, as new coatable weapons can be easily introduced by extending the abstract class without modifying existing code.

Implementing the Coating interface and defining concrete coating classes (SnowCoating, YewberryCoating) promotes modularity and extensibility, as new coatings can be added without changing the weapon classes. This design supports the Single Responsibility Principle, since each coating type independently defines its own behavior and effect logic. It also applies the Dependency Inversion Principle, as weapons depend on the Coating interface rather than specific coating implementations, making the system more flexible and loosely coupled.

The introduction of the StatusEffects class further enhances separation of concerns, ensuring that the Actor class is not overloaded with effect management responsibilities. This encapsulation improves maintainability and readability, while simplifying future additions of new status effects.

Overall, this design provides a clean, extensible, and maintainable architecture that follows both KISS and SOLID design principles. Coating and effect logic are clearly modularized, abstractions prevent redundancy, and the system remains easy to extend with minimal changes. The combination of abstraction, interfaces, and dedicated management classes results in a coherent design that balances simplicity with scalability, ensuring for future expansion of coating types and weapon functionalities.